

Red Hat OpenShift Data Foundation 4.22

Deploying OpenShift Data Foundation using bare metal infrastructure

Instructions on deploying OpenShift Data Foundation using local storage on bare metal infrastructure

Last Updated: 2026-09-24

Red Hat OpenShift Data Foundation 4.22 Deploying OpenShift Data Foundation using bare metal infrastructure

Instructions on deploying OpenShift Data Foundation using local storage on bare metal infrastructure

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Read this document for instructions about how to install Red Hat OpenShift Data Foundation to use local storage on bare metal infrastructure.

Table of Contents

PREFACE ... 3

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION ... 4

PREFACE ... 5

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION ... 6

CHAPTER 2. DEPLOY OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES ... 7
2.1. INSTALLING LOCAL STORAGE OPERATOR 7
2.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 7
2.3. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD 9
2.4. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE KUBERNETES AUTHENTICATION METHOD 9
2.5. CREATING OPENSHIFT DATA FOUNDATION CLUSTER ON BARE METAL 12

CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT ... 20
3.1. VERIFYING THE STATE OF THE PODS 20
3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY 22
3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY 23
3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST 23
3.5. VERIFYING THE MULTUS NETWORKING 24

CHAPTER 4. DEPLOY STANDALONE MULTICLOUD OBJECT GATEWAY ... 26
4.1. INSTALLING LOCAL STORAGE OPERATOR 26
4.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 26
4.3. CREATING A STANDALONE MULTICLOUD OBJECT GATEWAY 28

CHAPTER 5. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY ... 33

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION ... 34
6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE 34

PREFACE

Deploy OpenShift Data Foundation on OpenShift Container Platform using bare metal infrastructure and local storage devices to create internal cluster resources.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

PREFACE

Red Hat OpenShift Data Foundation supports deployment on existing Red Hat OpenShift Container Platform (RHOCP) bare metal clusters in connected or disconnected environments along with out-of-the-box support for proxy environments.

Both internal and external OpenShift Data Foundation clusters are supported on bare metal. See the Planning your deployment guide and the Preparing to deploy OpenShift Data Foundation chapter for more information about deployment requirements.

To deploy OpenShift Data Foundation, follow the appropriate deployment process based on your requirement:

- Internal mode – Deploy using local storage devices, or Deploy standalone Multicloud Object Gateway component
- External mode

Additional resources

- Planning your deployment
- Preparing to deploy OpenShift Data Foundation
- Deploy standalone Multicloud Object Gateway component
- External mode

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION

When you deploy OpenShift Data Foundation on OpenShift Container Platform using local storage devices, you can create internal cluster resources. This approach internally provisions base services so that all applications can access additional storage classes.

Before you begin the deployment of Red Hat OpenShift Data Foundation using local storage, ensure that you meet the resource requirements. See *Requirements for installing OpenShift Data Foundation using local storage devices*.

- Optional: If you want to enable cluster-wide encryption using an external Key Management System (KMS) follow these steps:
    - Ensure that you have a valid Red Hat OpenShift Data Foundation Advanced subscription. To know how subscriptions for OpenShift Data Foundation work, see *knowledgebase article on OpenShift Data Foundation subscriptions*.
    - When the Token authentication method is selected for encryption, refer to *Enabling cluster-wide encryption with the Token authentication using KMS*.
    - When the Kubernetes authentication method is selected for encryption then refer to *Enabling cluster-wide encryption with KMS using the Kubernetes authentication method* .
    - Ensure that you are using signed certificates on your vault servers.

After you have addressed the above, perform the following steps:

1. Install the Local Storage Operator.
2. Install the Red Hat OpenShift Data Foundation Operator.
3. Create the OpenShift Data Foundation cluster on bare metal.

Additional resources

- Requirements for installing OpenShift Data Foundation using local storage devices
- knowledgebase article on OpenShift Data Foundation subscriptions
- Enabling cluster-wide encryption with the Token authentication using KMS
- Enabling cluster-wide encryption with KMS using the Kubernetes authentication method

CHAPTER 2. DEPLOY OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES

2.1. INSTALLING LOCAL STORAGE OPERATOR

Install the Local Storage Operator from the Software Catalog before creating Red Hat OpenShift Data Foundation clusters on local storage devices.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Type **local storage** in the **Filter by keyword** box to find the **Local Storage Operator** from the list of operators, and click on it.
4. Set the following options on the **Install Operator** page:
    a. Update channel as **stable**.
    b. Installation mode as **A specific namespace on the cluster**.
    c. Installed Namespace as **Operator recommended namespace openshift-local-storage**.
    d. Update approval as **Automatic**.
5. Click **Install**.

Verification

- Verify that the Local Storage Operator shows a green tick indicating successful installation.

2.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
   a. Update Channel as **stable-4.22**.
   b. Installation Mode as **A specific namespace on the cluster**
   c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
   d. Select Approval Strategy as **Automatic** or **Manual**.
   If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

   If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
   e. Ensure that the **Enable** option is selected for the **Console plugin**.
   f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:

- Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
- Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

2.3. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD

You can enable the key value backend path and policy in the vault for token authentication.

Prerequisites

- Administrator access to the vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- Carefully, select a unique path name as the backend **path** that follows the naming convention since you cannot change it later.

Procedure

1. Enable the Key/Value (KV) backend path in the vault.
   For vault KV secret engine API, version 1:

   ```
   $ vault secrets enable -path=odf kv
   ```

   For vault KV secret engine API, version 2:

   ```
   $ vault secrets enable -path=odf kv-v2
   ```

2. Create a policy to restrict the users to perform a write or delete operation on the secret:

   ```
   echo '
   path "odf/*" {
     capabilities = ["create", "read", "update", "delete", "list"]
   }
   path "sys/mounts" {
   capabilities = ["read"]
   }'| vault policy write odf -
   ```

3. Create a token that matches the above policy:

   ```
   $ vault token create -policy=odf -format json
   ```

2.4. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE KUBERNETES AUTHENTICATION METHOD

You can enable the Kubernetes authentication method for cluster-wide encryption using the Key Management System (KMS).

Prerequisites

- Administrator access to Vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- The OpenShift Data Foundation operator must be installed from the Software Catalog.
- Select a unique path name as the backend **path** that follows the naming convention carefully. You cannot change this path name later.

Procedure

1. Create a service account:

```
$ oc -n openshift-storage create serviceaccount <serviceaccount_name>
```

where, ***<serviceaccount_name>*** specifies the name of the service account.

For example:

```
$ oc -n openshift-storage create serviceaccount odf-vault-auth
```

2. Create **clusterrolebindings** and **clusterroles**:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-
storage:_<serviceaccount_name>_
```

For example:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-storage:odf-vault-auth
```

3. Create a secret for the **serviceaccount** token and CA certificate.

```
$ cat <<EOF | oc create -f -
apiVersion: v1
kind: Secret
metadata:
  name: odf-vault-auth-token
  namespace: openshift-storage
  annotations:
    kubernetes.io/service-account.name: <serviceaccount_name>
type: kubernetes.io/service-account-token
data: {}
EOF
```

where, ***<serviceaccount_name>*** is the service account created in the earlier step.

4. Get the token and the CA certificate from the secret.

```
$ SA_JWT_TOKEN=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
{.data['token']}" | base64 --decode; echo)
$ SA_CA_CRT=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
```

```
{.data['ca\.crt']}" | base64 --decode; echo)
```

5. Retrieve the OCP cluster endpoint.

```
$ OCP_HOST=$(oc config view --minify --flatten -o jsonpath="{.clusters[0].cluster.server}")
```

6. Fetch the service account issuer:

```
$ oc proxy &
$ proxy_pid=$!
$ issuer="$( curl --silent http://127.0.0.1:8001/.well-known/openid-configuration | jq -r
.issuer)"
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

Or, for Vault version 1.21.0 and higher:

```
$ oc proxy &
$ proxy_pid=$!
$ ISSUER=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' \   |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token \   | jq -r
'.status.token' \   | cut -d . -f2 \   | base64 -d \   | jq '.iss')
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

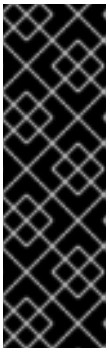
7. Use the information collected in the previous step to setup the Kubernetes authentication method in Vault:

```
$ vault auth enable kubernetes
```

```
$ vault write auth/kubernetes/config \
        token_reviewer_jwt="$SA_JWT_TOKEN" \
        kubernetes_host="$OCP_HOST" \
        kubernetes_ca_cert="$SA_CA_CRT" \
        issuer="$issuer"
```

Or, for Vault version 1.21.0 and higher:

```
$ oc exec -n vault -ti vault-0 – \
   vault write auth/kubernetes/config \
   token_reviewer_jwt="$SA_JWT_TOKEN" \
   kubernetes_host="$OCP_HOST" \
   kubernetes_ca_cert="$SA_CA_CRT" \
   issuer="$ISSUER"\
   token_audience="$AUDIENCE"
```



IMPORTANT

To configure the Kubernetes authentication method in Vault when the issuer is empty:

```
$ vault write auth/kubernetes/config \
        token_reviewer_jwt="$SA_JWT_TOKEN" \
        kubernetes_host="$OCP_HOST" \
        kubernetes_ca_cert="$SA_CA_CRT"
```

8. Enable the Key/Value (KV) backend path in Vault.
   For Vault KV secret engine API, version 1:

   ```
   $ vault secrets enable -path=odf kv
   ```

   For Vault KV secret engine API, version 2:

   ```
   $ vault secrets enable -path=odf kv-v2
   ```

9. Create a policy to restrict the users to perform a **write** or **delete** operation on the secret:

   ```
   echo '
   path "odf/*" {
     capabilities = ["create", "read", "update", "delete", "list"]
   }
   path "sys/mounts" {
   capabilities = ["read"]
   }'| vault policy write odf -
   ```

10. Generate the roles:

    ```
    $ vault write auth/kubernetes/role/odf-rook-ceph-op \
          bound_service_account_names=rook-ceph-system,rook-ceph-osd,noobaa \
          bound_service_account_namespaces=openshift-storage \
          policies=odf \
          ttl=1440h
    ```

    The role **odf-rook-ceph-op** is later used while you configure the KMS connection details during the creation of the storage system.

    ```
    $ vault write auth/kubernetes/role/odf-rook-ceph-osd \
          bound_service_account_names=rook-ceph-osd \
          bound_service_account_namespaces=openshift-storage \
          policies=odf \
          ttl=1440h
    ```

2.5. CREATING OPENSHIFT DATA FOUNDATION CLUSTER ON BARE METAL

Prerequisites

Create an OpenShift Data Foundation storage cluster on bare metal infrastructure by using the OpenShift Web Console wizard to define local SSD devices, configure capacity and performance profiles, and optionally enable encryption and Multus networking.

- Ensure that the disk type is SSD, which is the only supported disk type.
- If you want to use the multi network plug-in (Multus), before deployment you must create network attachment definitions (NADs) that is later attached to the cluster. For more information, see Multi network plug-in (Multus) support and Creating network attachment definitions.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation → Storage Systems → Create StorageSystem**.
2. In the **Backing storage** page, select the **Create a new StorageClass using the local storage devices** option and click **Next**.
3. In the **Optional settings** page, you can configure the following optional items:
   - **Enable Network Files System**
     Automatically enable NFS for the cluster
   - **Use Ceph RBD as the default StorageClass**
     Sets Ceph RBD as the default StorageClass, eliminating the need to manually annotate a StorageClass.
   - **Set default StorageClass for virtualization**
     Marks the RBD virtualization storage class as the default for KubeVirt VM disks (PVs) after installation.
   - **Use external PostgreSQL** [Technology preview]
     Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.

     IMPORTANT

     OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

     As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

     If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

     Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

     a. Provide the following connection details:

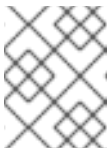
- **Username**
- **Password**
- **Server name** and **Port**
- **Database name**

b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.

- **Enable automatic backup**
  Allows automatic backup for the Multicloud Object Gateway metadata database.

a. Select the backup frequency: Daily, Weekly, or Monthly.

b. Specify the number of backups to retain.



NOTE

This option is disabled if the **Use external PostgreSQL** option is selected.

c. Click **Next**.

4. In the **Create local volume set** page, provide the following information:

a. Enter a name for the **LocalVolumeSet** and the **StorageClass**.
The local volume set name appears as the default value for the storage class name. You can change the name.

b. Select one of the following:

- **Disks on all nodes**
  Uses the available disks that match the selected filters on all the nodes.
- **Disks on selected nodes**
  Uses the available disks that match the selected filters only on the selected nodes.



IMPORTANT

- The flexible scaling feature is enabled only when the storage cluster that you created with three or more nodes are spread across fewer than the minimum requirement of three availability zones.
  For information about flexible scaling, see knowledgebase article on *Scaling OpenShift Data Foundation cluster using YAML when flexible scaling is enabled*.
- Flexible scaling features get enabled at the time of deployment and can not be enabled or disabled later on.
- If the nodes selected do not match the OpenShift Data Foundation cluster requirement of an aggregated 30 CPUs and 72 GiB of RAM, a minimal cluster is deployed.
  For minimum starting node requirements, see the Resource requirements section in the *Planning* guide.

c. From the available list of **Disk Type**, select **SSD/NVMe**.

d. Expand the **Advanced** section and set the following options:

Volume Mode	Block is selected as the default value.
Device Type	Select one or more device types from the dropdown list.
Disk Size	Set a minimum size of 100GB for the device and maximum available size of the device that needs to be included.
Maximum Disks Limit	This indicates the maximum number of Persistent Volumes (PVs) that you can create on a node. If this field is left empty, then PVs are created for all the available disks on the matching nodes.

e. Click **Next**.
A pop-up to confirm the creation of LocalVolumeSet is displayed.

f. Click **Yes** to continue.

5. In the **Capacity and nodes** page, configure the following:

a. **Available raw capacity** is populated with the capacity value based on all the attached disks associated with the storage class. This takes some time to show up. The **Selected nodes** list shows the nodes based on the storage class.

b. In the **Configure performance** section, select one of the following performance profiles:

- Lean
Use this in a resource constrained environment with minimum resources that are lower than the recommended. This profile minimizes resource consumption by allocating fewer CPUs and less memory.

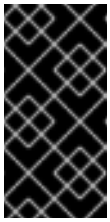
- Balanced (default)
Use this when recommended resources are available. This profile provides a balance between resource consumption and performance for diverse workloads.

- Performance
Use this in an environment with sufficient resources to get the best performance. This profile is tailored for high performance by allocating ample memory and CPUs to ensure optimal execution of demanding workloads.



NOTE

You have the option to configure the performance profile even after the deployment using the **Configure performance** option from the options menu of the **StorageSystems** tab.



IMPORTANT

Before selecting a resource profile, make sure to check the current availability of resources within the cluster. Opting for a higher resource profile in a cluster with insufficient resources might lead to installation failures.

For more information about resource requirements, see Resource requirement for performance profiles.

c. Optional: Select the **Taint nodes** checkbox to dedicate the selected nodes for OpenShift Data Foundation.

d. Click **Next**.

6. Optional: In the **Advanced settings** page, configure the following based on your requirement:

a. Select **Enable forceful deployment** to override and completely wipe any data on disks used to create this storage cluster and type **CONFIRM** in the text box.



WARNING

Any data on the disks used to create this storage cluster will be deleted. This cannot be undone. Use this option only when you are certain that the disks do not contain any data you need to preserve, such as when reusing disks from a previous OpenShift Data Foundation installation or when disks contain data from other Ceph clusters.

b. Select **Use erasure coding** to enable erasure coding instead of replication to protect the cluster data.
Erasure coding is supported for Block (RBD), File (CephFS), and RADOS Gateway (RGW) object storage.

For more information, see Erasure coding considerations.

a. Click **Next**.

1. Optional: In the **Security and network** page, configure the following based on your requirement:

b. To enable encryption, select **Enable data encryption for block and file storage**

c. Select one or both of the following **Encryption level**:

- **Cluster-wide encryption**
Encrypts the entire cluster (block and file).

- **StorageClass encryption**
Creates encrypted persistent volume (block only) using encryption enabled storage class.

d. Optional: Select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

i. From the **Key Management Service Provider** drop-down list, select one of the following providers and provide the necessary details:

- **Vault**

A. Select an **Authentication Method**.

- **Using Token authentication method**
  - Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
  - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
  - Optional: Enter **TLS Server Name** and **Vault Enterprise Namespace**.
  - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
  - Click **Save**.
- **Using Kubernetes authentication method**
  - Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.
  - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
    - Optional: Enter **TLS Server Name** and **Authentication Path** if applicable.
    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key**.
  - Click **Save**.



NOTE

In case you need to enable key rotation for Vault KMS, run the following command in the OpenShift web console after the storage cluster is created:

```
oc patch storagecluster ocs-storagecluster -n openshift-storage --type=json -p '[{"op": "add", "path":"/spec/encryption/keyRotation/enable", "value": true}]'
```

- **Thales CipherTrust Manager (using KMIP)**

A. Enter a unique **Connection Name** for the Key Management service within the project.

B. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

- **Address**: 123.34.3.2
- **Port**: 5696

C. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

D. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

E. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

ii. Select a **Network**.

e. Select one of the following:

- **Default (SDN)**
  If you are using a single network.
- **Custom (Multus)**
  If you are using multiple network interfaces.

i. Select a **Public Network Interface** from the dropdown.

ii. Select a **Cluster Network Interface** from the dropdown. For information about Multus architecture, see link:



NOTE

If you are using only one additional network interface, select the single **NetworkAttachmentDefinition**, that is,**ocs-public-cluster** for the Public Network Interface and leave the Cluster Network Interface blank.

f. Click **Next**.

1. In the **Review and create** page, review the configuration details.
   To modify any configuration settings, click **Back** to go back to the previous configuration page.

2. Click **Create StorageSystem**.



NOTE

When your deployment has five or more nodes, racks, or rooms, and when there are five or more number of failure domains present in the deployment, you can configure Ceph monitor counts based on the number of racks or zones. An alert is displayed in the notification panel or Alert Center of the OpenShift Web Console to indicate the option to increase the number of Ceph monitor counts. You can use the **Configure** option in the alert to configure the Ceph monitor counts. For more information, see Resolving low Ceph monitor count alert .

Verification

- To verify the final Status of the installed storage cluster:

  a. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System → ocs-storagecluster**.

  b. Verify that the **Status** of the **StorageCluster** is **Ready** and has a green tick mark next to it.

- To verify if the flexible scaling is enabled on your storage cluster, perform the following steps (for arbiter mode, flexible scaling is disabled):

  1. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System**

  2. From the Action menu, select **Edit Storage System**.

  3. In the YAML tab, search for the keys **flexibleScaling** in the **spec** section and **failureDomain** in the **status** section. If **flexible scaling** is true and **failureDomain** is set to host, flexible scaling feature is enabled:

     ```
     spec:
     flexibleScaling: true
     […]
     status:
     failureDomain: host
     ```

- To verify that all the components for OpenShift Data Foundation are successfully installed, see Verifying your OpenShift Data Foundation installation .

- To verify the multi networking (Multus), see Verifying the Multus networking.

Additional resources

- Scaling Storage guide

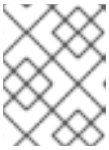
CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT

3.1. VERIFYING THE STATE OF THE PODS

Verify that all required OpenShift Data Foundation pods are in a **Running** state after deployment.

Procedure

1. Click **Workloads → Pods** from the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

For more information on the expected number of pods for each component and how it varies depending on the number of nodes, see the following table:

1. Set filter for Running and Completed pods to verify that the following pods are in **Running** and **Completed** state:



NOTE

The available pods depend on the cluster configuration. When the cluster is deployed as a standalone Multicloud Object Gateway, the **rook-ceph-operator-*** pods are not available. Similarly, when the cluster is deployed without the Multicloud Object Gateway, **noobaa-*** pods are not available.

Pods corresponding to OpenShift Data Foundation cluster

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node)

Component	Corresponding pods
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)
MON	**rook-ceph-mon-*** (3 pods distributed across storage nodes)
MGR	**rook-ceph-mgr-*** (2 pods on different storage nodes, one active, one standby)
MDS	**rook-ceph-mds-ocs-storagecluster-cephfilesystem-*** (2 pods distributed across storage nodes)
RGW	**rook-ceph-rgw-ocs-storagecluster-cephobjectstore-*** (1 pod on any storage node)

Component	Corresponding pods
CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on each storage node)
rook-ceph-crashcollector	**rook-ceph-crashcollector-*** (1 pod on each storage node)
OSD	• **rook-ceph-osd-*** (1 pod for each device)

3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY

Verify that the OpenShift Data Foundation cluster is healthy after installation.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

3. In the **Status** card of the **Block and File** tab, verify that the *Storage Cluster* has a green tick.

4. In the **Details** card, verify that the cluster information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the **Block and File** dashboard, see Monitoring OpenShift Data Foundation.

3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY

Verify that the Multicloud Object Gateway is healthy after deployment to confirm that the object service and data resiliency are operational and that the NooBaa database has a recent backup.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.

   b. In the **Details** card, verify that the MCG information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the object service dashboard, see Monitoring OpenShift Data Foundation.



IMPORTANT

To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up your NooBaa DB, follow the steps in the knowledgebase article, Perform a One-Time Backup of the Database for the Multicloud Object Gateway.

3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST

Verify that the expected OpenShift Data Foundation storage classes are present after deployment.

Procedure

1. Click **Storage → Storage Classes** from the left pane of the OpenShift Web Console.

2. Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:

   - **ocs-storagecluster-ceph-rbd**
   - **ocs-storagecluster-cephfs**
   - **openshift-storage.noobaa.io**
   - **ocs-storagecluster-ceph-rgw**

3.5. VERIFYING THE MULTUS NETWORKING

To determine if Multus is working in your cluster, verify the Multus networking.

Procedure

1. Based on your Network configuration choices, the OpenShift Data Foundation operator will do one of the following:
   - If only a single NetworkAttachmentDefinition (for example, **ocs-public-cluster**) was selected for the Public Network Interface, then the traffic between the application pods and the OpenShift Data Foundation cluster will happen on this network. Additionally the cluster will be self configured to also use this network for the replication and rebalancing traffic between OSDs.
   - If both NetworkAttachmentDefinitions (for example, **ocs-public** and **ocs-cluster**) were selected for the Public Network Interface and the Cluster Network Interface respectively during the Storage Cluster installation, then client storage traffic will be on the public network and cluster network for the replication and rebalancing traffic between OSDs.
2. To verify the network configuration is correct, complete the following:
3. In the OpenShift console, navigate to **Storage → Data Foundation → Storage System**.
4. From the Action menu, select **Edit Storage System**.
5. In the YAML tab, search for **network** in the **spec** section and ensure the configuration is correct for your network interface choices. This example is for separating the client storage traffic from the storage replication traffic.
   Sample output:

```
[..]
spec:
  [..]
  network:
    ipFamily: IPv4
    provider: multus
    selectors:
      cluster: openshift-storage/ocs-cluster
      public: openshift-storage/ocs-public
  [..]
```

1. To verify the network configuration is correct using the command line interface, run the following commands:

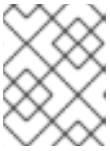
```
$ oc get storagecluster ocs-storagecluster \
-n openshift-storage \
-o=jsonpath='{.spec.network}{"\n"}'
```

Sample output:

```
{"ipFamily":"IPv4","provider":"multus","selectors":{"cluster":"openshift-storage/ocs-cluster","public":"openshift-storage/ocs-public"}}
```

1. **Confirm the OSD pods are using correct network**

In the **openshift-storage** namespace use one of the OSD pods to verify the pod has connectivity to the correct networks. This example is for separating the client storage traffic from the storage replication traffic.



NOTE

Only the OSD pods will connect to both Multus public and cluster networks if both are created. All other OCS pods will connect to the Multus public network.

```
$ oc get -n openshift-storage $(oc get pods -n openshift-storage -o name -l app=rook-ceph-osd | grep 'osd-0') -o=jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-status}{"\n"}'
```

Sample output:

```
[{
    "name": "openshift-sdn",
    "interface": "eth0",
    "ips": [
        "10.129.2.30"
    ],
    "default": true,
    "dns": {}
},{
    "name": "openshift-storage/ocs-cluster",
    "interface": "net1",
    "ips": [
        "192.168.2.1"
    ],
    "mac": "e2:04:c6:81:52:f1",
    "dns": {}
},{
    "name": "openshift-storage/ocs-public",
    "interface": "net2",
    "ips": [
        "192.168.1.1"
    ],
    "mac": "ee:a0:b6:a4:07:94",
    "dns": {}
}]
```

To confirm the OSD pods are using correct network using the command line interface, run the following command (requires the jq utility):

```
$ oc get -n openshift-storage $(oc get pods -n openshift-storage -o name -l app=rook-ceph-osd | grep 'osd-0') -o=jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-status}{"\n"}' | jq -r '.[].name'
```

Sample output:

```
openshift-sdn
openshift-storage/ocs-cluster
openshift-storage/ocs-public
```

CHAPTER 4. DEPLOY STANDALONE MULTICLOUD OBJECT GATEWAY

4.1. INSTALLING LOCAL STORAGE OPERATOR

Install the Local Storage Operator from the Software Catalog before creating Red Hat OpenShift Data Foundation clusters on local storage devices.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Type **local storage** in the **Filter by keyword** box to find the **Local Storage Operator** from the list of operators, and click on it.
4. Set the following options on the **Install Operator** page:
    a. Update channel as **stable**.
    b. Installation mode as **A specific namespace on the cluster**.
    c. Installed Namespace as **Operator recommended namespace openshift-local-storage**.
    d. Update approval as **Automatic**.
5. Click **Install**.

Verification

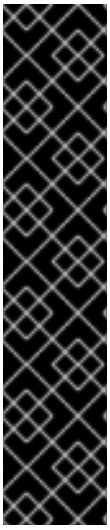
- Verify that the Local Storage Operator shows a green tick indicating successful installation.

4.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
   a. Update Channel as **stable-4.22**.
   b. Installation Mode as **A specific namespace on the cluster**
   c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
   d. Select Approval Strategy as **Automatic** or **Manual**.
   If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

   If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
   e. Ensure that the **Enable** option is selected for the **Console plugin**.
   f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:

- Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
- Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

4.3. CREATING A STANDALONE MULTICLOUD OBJECT GATEWAY

You can create only the standalone Multicloud Object Gateway (MCG) component while deploying OpenShift Data Foundation. After you create the MCG component, you can create and manage buckets using the MCG object browser. For more information, see Creating and managing buckets using MCG object browser.

Prerequisites

- Ensure that the OpenShift Data Foundation Operator is installed.
- The Multicloud Object Gateway database uses ReadWriteOncePod (RWOP) access mode to safeguard against node failures. If the underlying CSI driver does not support RWOP, the access mode must be manually changed to ReadWriteOnce (RWO) before deployment. For more information, see the knowledgebase article How to change NooBaa DB access mode from RWOP to RWO in standalone MCG deployments?.
- Ensure that the CSI driver is configured with **fsGroupPolicy: File**. If **fsGroupPolicy: None** is used, volume permissions are not set automatically, which can cause PostgreSQL pod initialization failures.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → StorageCluster**.
2. On the Welcome page, click Configure Data Foundation.
3. Select Setup Multicloud Object Gateway.
4. In the **Backing storage** page, select the following:
5. In the **Advanced Settings** page, you can configure the following optional items:
   - **Use external PostgreSQL** [Technology preview]
     Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.



IMPORTANT

OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

a. Provide the following connection details:

- **Username**
- **Password**
- **Server name** and **Port**
- **Database name**

b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.

- **Enable automatic backup**
  Allows automatic backup for the NooBaa database.

a. Select the backup frequency: Daily, Weekly, or Monthly.

b. Specify the number of backups to retain.

c. Select a VolumeSnapshotClass to add a volume snapshot.



NOTE

This option is disabled if the **Use external PostgreSQL** option is selected.

d. Click **Next**.

6. In the **Security** page, select any of the following optional items:

a. Select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

b. From the **Key Management Service Provider** drop-down list, either select **Vault** or **Thales CipherTrust Manager (using KMIP)**. If you selected **Vault**, go to the next step. If you selected **Thales CipherTrust Manager (using KMIP)**, go to step iii.

c. Select an **Authentication Method**.

Using Token authentication method

- Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
    - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace**.
    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
    - Click **Save** and skip to step iv.

Using Kubernetes authentication method

- Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.
- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
    - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace** if applicable.
    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
    - Click **Save** and skip to step iv.

d. To use **Thales CipherTrust Manager (using KMIP)** as the KMS provider, follow the steps below:

i. Enter a unique **Connection Name** for the Key Management service within the project.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

- **Address**: 123.34.3.2
- **Port**: 5696

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

e. Select a **Network**.

f. Click **Next**.

7. In the **Review and create** page, review the configuration details:
   To modify any configuration settings, click **Back**.

8. Click **Create StorageSystem**.

Verification

Verifying that the OpenShift Data Foundation cluster is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.

2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.

   b. In the **Details** card, verify that the MCG information is displayed.

Verifying the state of the pods

1. Click **Workloads → Pods** from the OpenShift Web Console.

2. Select **openshift-storage** from the **Project** drop-down list and verify that the following pods are in **Running** state.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)

Component	Corresponding pods
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

CHAPTER 5. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY

The topology provides a visual representation of the OpenShift Data Foundation storage cluster at multiple levels of abstraction and enables interaction with each layer. It illustrates how the cluster components are organized and related, while offering visibility into Object Storage Devices (OSDs), including their count, type, and status.

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Data Foundation → Topology**.
   The view shows the storage cluster and the topology groupings inside it. You can see the nodes depicted by circular entities within the topology groupings, which are indicated by dotted lines. The topology groupings are based on the cluster's topology configuration (for example, racks in bare metal or vSphere clusters, or zones in cloud-based clusters). The label of each item or resource contains basic information such as status and health or indication for alerts.

2. Choose a node to view node details on the right-hand panel. You can also access resources or deployments within a node by clicking on the search/preview decorator icon.
   The node details panel displays information about the OSDs running on that node, including:

   - Number of OSDs on the node
   - OSD disk type (for example, SSD, HDD)
   - OSD status (for example, Running, Error, CrashLoopBackOff)

3. To view deployment details

   a. Click the preview decorator on a node. A modal window appears above the node that displays all of the deployments associated with that node along with their statuses.

   b. Click the **Back to main view** button in the model's upper left corner to close and return to the previous view.

   c. Select a specific deployment to see more information about it. All relevant data is shown in the side panel.

4. Click the **Resources** tab to view the pods information. This tab provides a deeper understanding of the problems and offers granularity that aids in better troubleshooting.

5. Click the **OSD information** tab to view detailed information about the OSDs. This tab provides more comprehensive OSD-specific details, including:

   - OSD pod name and status
   - Disk device associated with each OSD
   - Device class (for example, SSD, HDD)
   - Resource utilization metrics

6. Click the pod links to view the pod information page on OpenShift Container Platform. The link opens in a new window.

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION

6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE

Uninstalling OpenShift Data Foundation in internal mode permanently removes the storage cluster, all associated PVCs, and the operator.

Procedure

1. To uninstall OpenShift Data Foundation in internal mode, refer to the knowledgebase article on Uninstalling OpenShift Data Foundation.